Real-World Column Dictionary Improvement

Antigravity ke liye phased, approval-gated engineering instructions

Purpose: SENOVA ko real business CSV/XLSX headers zyada reliably recognise karwana, bina invented keywords, forced mappings, privacy risk, ya existing calculations ko break kiye.

Non-negotiable execution model
No fixed calendar timeline. Complete Phase 1, report evidence, wait for the user's explicit approval, and only then begin Phase 2. Repeat for every phase. Each phase must be testable and reversible.

Current user-supplied materials: original detailed PDF playbook, SENOVA real-world header starter pack ZIP, and other business sample/header packs the user uploads. Treat each attachment as an input to inspect—not as proof that every source file has already been verified.

Prepared: 11 October 2026 | Language: Hinglish instructions with technical terms retained

Quick start: Antigravity ko kya karna hai

Is PDF ko execution contract samjho. Antigravity ko sab phases ek saath implement karne ki permission nahi hai. Sirf current phase implement hoga, phir test report aur review output diya jayega. User ke “Approve Phase N” kehne ke baad hi next phase shuru hoga.

User ke decisions jo ab fixed hain

- Source-backed words only: AI-generated, guessed, or fabricated aliases production dictionary mein add nahi karne.
- Attachments-first: jo PDFs, ZIPs, CSV/XLSX templates aur starter packs user ne diye hain, unhe pehle inspect karo. Unhi mein se actual content use karo; baar-baar wahi files internet se dhoondhne ya redownload karne ki zaroorat nahi.
- External sources only for gaps: supplied files se coverage na mile tab official source pages/templates open karo. Login/paywall/access control bypass nahi karna.
- Approval gates: Phase 1 -> report -> user tests/approves -> Phase 2. Same for Phase 2 -> 3 -> 4.
- No timeline-driven work: day/hour schedule nahi. Work ready tab complete hoga jab phase acceptance criteria pass hon. Fast tasks ko artificially stretch mat karo.
- Required upload fields: only Date plus one valid Amount path are compulsory. Category, Item, Cost Price and all other business dimensions are optional for upload/analysis confirmation.
- Numbers remain deterministic: Pandas/NumPy calculate; dictionary/embedding/Gemini only classify or suggest column roles. Never invent calculations or formulas.
- Git safety: no commit, push, branch merge or PR creation without explicit permission. Preserve user changes; create a backup/branch locally where allowed.

The two required conditions

Required	Accepted path	Validation
Date	A column confidently mapped to transaction/sales date, subject to current date ambiguity policy.	Must contain parseable dates at the project's existing acceptance threshold; report invalid ratio.
Amount path A	Line Total / sales amount column mapped.	Use existing approved line-amount semantics; do not assume invoice total is a per-row amount.
Amount path B	Both Quantity AND Selling Price mapped.	Revenue amount = Quantity × Selling Price calculated by the existing Pandas/NumPy calculation layer.

If Date + A or Date + B is satisfied, “Confirm & analyse” must be enabled. Do not require Category, Item, Cost Price, Discount, Tax, MRP, Stock On Hand or any other dimension for confirmation.

Execution governance and deliverables

Before any code changes

- Read this PDF end-to-end and list all attachments visible in the current workspace.
- Inspect repository structure, current branch, git status, current aliases/dictionary, semantic catalog, Tier 1, FastEmbed, Tier 2 Gemini path, privacy masking, UI mapping flow, query schema and existing tests.
- Record actual counts separately: source-backed dictionary records, normalized unique phrases, alias map entries, fuzzy entries, semantic catalog synonyms, duplicate phrases and role conflicts. Never assume “286 fuzzy keywords” is the total dictionary size.
- Run existing relevant suites and baseline fixtures before modifying behavior. Save exact commands, output, pass/fail/skip counts and environment limitations.
- Make a file-by-file plan for only the current phase. Minimal additive changes; maintain line endings and naming. No broad refactors unless demonstrably required.

At the end of each phase, return all of this

Deliverable	Required content
Phase summary	What changed, what intentionally did not change, changed files with repo-relative paths.
Evidence report	Source/file used, exact header spellings, module/industry, URLs or archive paths, status of every proposed entry.
Dictionary audit	Added, duplicate, conflict, ambiguous, rejected and still-unverified counts.
Tests	Exact commands, actual terminal output, before/after counts, regression values and limitations.
CHANGES.md	Design notes, requirement-to-files-to-tests table, compatibility notes, and any existing test assertion changed with justification. Never delete or weaken tests.
Patch delivery	If GitHub writing is unavailable, ZIP only modified/new files at their original repository-relative paths. Include CHANGES.md and apply instructions.
Approval question	Explicitly ask user to test/review Phase N. Do not proceed automatically to Phase N+1.

If a source or tool is unavailable

- Document the exact source URL or attachment name, the attempted access, the limitation and whether the entry remains unverified.
- Continue with other accessible supplied materials; do not block the entire phase for a single inaccessible source.
- Never claim a file was downloaded, opened, inspected, or tested unless it actually was.
- If the attachment itself is unavailable, report it precisely; do not invent its contents or pretend to have read it.

Phase 1 — Baseline, supplied assets and two-field upload rule

Goal: establish a trustworthy baseline, use the provided real-world header packs, remove the incorrect five-required-fields gate, and preserve columns that are not part of calculations. Do only this phase until user approval.

1.1 Inspect the assets already supplied

- Open and inventory the original dictionary-improvement PDF and the real-world starter pack ZIP attached by the user.
- List every file in each archive, file type, row/header count and whether it is raw evidence, a proposed header list, documentation, code or instructions.
- Inspect other user-provided “normal base book”, “all sales pack” and source-template attachments when present. Use the actual names found in the workspace; do not assume those names or contents if they are not available.
- Read all supplied CSV/XLSX header rows only for dictionary collection. Avoid importing real transaction values unless a test specifically needs them; do not store personal/business values in the dictionary.
- The starter pack contains documented examples, including some fields whose exact literal export header has not yet been validated. Preserve verification status and do not blindly promote candidate role suggestions into production aliases.

1.2 Baseline audit

- Create a baseline file or machine-readable report of the current dictionary structures and role counts before touching them.
- For representative uploaded datasets, record each column header, dictionary match, embedding decision, Gemini route/source, final canonical role, final UI visibility and whether it was correctly mapped.
- Separate true misses from intentionally stored-only fields such as personal columns, ID-like identifiers, free text and unresolved ambiguous columns.
- Include electronics regression fixture testing2/03_electronics_shopify_orders.csv and record expected values: Revenue 5,757,679; Cost 3,450,780; Profit 2,306,899; Units 2,621; Discount Given 1,211,000; Discount % 17.38. Existing tests/data may establish units/rounding; do not change formulas merely to force these values.

1.3 Change required-column validation

- Find all frontend and backend places that hard-require the fixed five fields (Category, Item, Date, Cost Price, Amount or equivalent). Update both client and server validation; fixing only the button UI is insufficient.
- Required condition: Date + Line Total, OR Date + Quantity + Selling Price.
- Cost Price missing: analysis must still start; profit, margin, COGS and profit-based insights/lines are unavailable with a reason, not zero.
- Item missing: hide item rankings, item insights and inventory views with a reason. Category missing: hide category views with a reason.
- Quantity missing when Line Total is provided: revenue trends and revenue KPIs still work; units and average price are unavailable with a reason.
- Keep unknown distinct from zero. Use the product’s existing unavailable wording pattern, preferably “Not available: needs ” where that is the existing convention.
- The mapping screen lists only the missing required condition under “Still need a column for”. Add a non-blocking hint such as “Add Cost Price to unlock profit and margin”.

1.4 Extra columns: deterministic classification

Class	Rule	Behaviour
Date-like	Parse ratio >= 0.80	Preserve as stored-only unless it has a verified approved role. Do not silently choose sale date if several date columns exist.
ID-like	Header contains id/no/number/code/ref; OR integer-looking unique ratio >= 0.90; OR constant column	Stored-only; not a measure, filter or chart axis by default. Preserve leading zeroes.
Personal / PII	Email/phone pattern; use existing PII mask helpers; name-like/unidentified value protection applies for Gemini too	Stored-only; never offered as filter/axis; never send sample values to Gemini.
Numeric	A safe numeric conversion can be measured	Extra measure; default sum. If header contains %, pct, percent, rate, ratio or margin, default average and UI must say “average, not sum”.
Low-cardinality text	At most 50 distinct values OR unique ratio <= 0.20	Extra dimension if not personal/ID-like/ambiguous.
Other text	Does not fit above	Stored-only with reason.

Cap extra columns at 30 per file. Keep original header as display label; internal keys must be safe deterministic names such as x_dim: and x_meas:, with collision handling for duplicate/odd headers. Never generate code, formulas or SQL from raw header text.

1.5 Per-file registry and UI

- Replace hard-coded closed lists only as needed for per-file extra-column registry support. Built-in keys remain supported.
- API schema for a file should return its extra registry, class, display label, aggregation and safe key. Query endpoints accept built-in keys or keys in that specific file registry only; unknown or cross-file key should yield 422.
- Frontend Chart Studio, chart axes and filters merge canonical built-in options with that file’s registry. Personal, ID-like and stored-only columns are not offered as filter/axis.
- Coverage panel lists every uploaded column: source/original header, role/class, how used, and reason if stored-only. Add a compact count summary and expandable details. Reuse existing neutral “Recognised, not analysed” badge where available.
- Short coverage summary also appears in “What this file can answer”.

Phase 1 tests / acceptance gate

- Date + Line Total only: upload confirmation enabled and revenue analysis works.
- Date + Quantity + Selling Price only: confirmation enabled and revenue calculated through current engine.
- No Cost / Category / Item: no false blockers; dependent profit/category/item/inventory outputs become unavailable/hidden with reasons, never fake zero.
- Fixture with extras: Delivery Charge sum 22,480; GST % average 15.05; City has 4 distinct values; Sales Rep has 3; Order ID is ID-like; Notes stored-only; Customer Phone personal/stored-only.
- Dynamic key validation: unknown key rejected; a key belonging to a different file rejected.
- Duplicate/odd headers, 30-column cap, safe slugs, percent/rate aggregation average, and no PII filter/axis options tested.
- Electronics regression values exactly preserved. Full backend pytest and frontend Vitest suites pass before phase is presented as ready.

Phase 1 exit: provide report/ZIP, exact test output and manual test steps. STOP. User tests and approves before Phase 2 begins.

Phase 2 — Ambiguous headers and safe mapping memory

Start condition: user explicitly approved Phase 1. Phase 2 must not change Phase 1 acceptance behavior except where a documented bug is found and tested.

2.1 Deterministic evidence rules

Header family	Candidate roles	Rules before automatic decision
Rate / Price / Unit Price	unit_price, tax_rate, discount_pct	If Quantity × value matches a known amount column in >=90% of comparable rows, unit_price is supported. If all values are GST slabs {0,5,12,18,28} or decimal equivalents {0.05,0.12,0.18,0.28}, consider tax_rate. If 0-100 and a discount-named neighbour exists, consider discount_pct. If multiple candidates remain, ask.
Amount / Total / Value	line_total, invoice_total, payment_amount, tax_amount	If Quantity × Rate matches, consider line_total. If identical for all rows of each Invoice No, consider invoice_total and never sum repeated line-level copies. Bank/Dr/Cr neighbours may support payment/ledger interpretation; remain ask if ambiguous.
Name	item, customer, salesperson	Phone/Address/Email/GSTIN neighbours support customer; Item Code/SKU/HSN support item. Distinct-value ratio may help but cannot alone prove role.
Multiple date fields	sale/order/invoice/bill/voucher date vs due/ship/delivery/expiry date	Prefer transaction-related date only with clear evidence. If unclear which is sale date, ask once in grouped dialog; never silently use due/delivery/expiry date.

Every ambiguous column yields decision=auto only if exactly one candidate passes a documented strong test. Otherwise decision=ask. Thresholds and reasons must be observable in tests and logs without logging personal values.

2.2 Grouped clarification dialog

- One grouped dialog for all open header questions, not one popup per column.
- Show only masked/minimal sample representations and 2–3 role choices from existing fixed allowed roles; include “Something else (type a note)” and “Skip for now”.
- Skip never blocks analysis if Phase 1 requirements (Date + valid Amount path) are met.
- A user choice overrides an uncertain automated suggestion for that upload; show a clear label explaining the source of mapping.

2.3 Mapping memory

- Key mappings by a hash of normalized header set plus individual normalized header, scoped to owner/user. Use existing per-user storage convention if available; otherwise local JSON with TTL and owner ID may be used and documented.
- Store header names and approved roles only; never values, examples, or PII samples.
- On repeat upload, auto-apply eligible saved choices with label “from your earlier choice”; allow the user to change them.
- Add ownership/security tests so another user cannot read or overwrite mappings.

2.4 Invoice total repeat guard

- Create test file where one invoice total repeats on 5 item rows. Revenue must not be counted five times.
- Use existing line-level amount if available; otherwise aggregate invoice-level total once per invoice or mark the role ambiguous/ask. Never sum repeated invoice totals row-wise.
- Test multiple invoices, different totals, missing invoice identifiers and mixed line/invoice total candidates.

Phase 2 exit tests

- Unit tests for each Rate/Price, Amount/Total, Name and multiple-date rule, including positive and ambiguous/negative cases.

- Grouped dialog renders for multiple questions and only one dialog is opened; skip does not block valid analysis.
- Mapping memory round-trip, owner isolation, TTL and “earlier choice” UI label.
- Five repeated invoice rows guard; no revenue inflation.
- All Phase 1 tests remain green. Full backend and frontend suites pass. Produce report/ZIP and STOP for explicit approval.

Phase 3 — Offline-first, source-backed dictionary

Start condition: explicit Phase 2 approval. Focus on converting the existing aliases and semantic synonyms to an auditable offline dictionary without changing current mapping results.

3.1 Dictionary source file and schema

- Proposed file: backend/app/data/column_dictionary.json or a compatible existing location if current architecture already has a data registry.
- Fields: phrase (normalized); role; language (en, hi, kn, hinglish, other); industry (general, garment, pharmacy, grocery, restaurant, electronics, e-commerce, wholesale, accounting, manufacturing, services, other); source (builtin, tally, erpnext, shopify, zoho, odoo, square, uci, kaggle, github, data.gov.in, user_confirmed); source_url; source_type; ambiguous; candidates; prior only if current scoring truly uses it; verification_status; evidence_note; collected_at.
- Keep exact observed raw header spellings and multiple provenance links when useful. No customer values are allowed in the dictionary.
- Role must be from the repo’s current fixed allowed role set; industry or source context does not authorize a new canonical role.

3.2 Seed without regression

- Read current alias map and semantic catalog synonyms and take a deterministic snapshot of phrase -> current role before migration.
- Migrate/seed existing phrases; verify every prior alias resolves to the same role after loading the new dictionary. Add snapshot tests proving no behavior drift for existing aliases.
- Keep ambiguous phrases ambiguous; do not promote candidate examples from starter CSV unless the role is verified and supported by existing allowed roles.
- Dictionary hit never receives Gemini route. If AI is off and FastEmbed is unavailable, dictionary-backed headers still resolve.

3.3 Normalization, exact and bounded fuzzy lookup

- Strip BOM; trim; Unicode NFKC; lowercase/casefold; collapse whitespace and apply documented punctuation rules; preserve Devanagari and Kannada script.
- Lookup order: (1) offline dictionary/alias exact lookup, (2) bounded fuzzy only after exact lookup, (3) FastEmbed similarity for unresolved headers, (4) Gemini only unresolved after explicit consent, privacy masking and schema-bounded request, (5) user note/skip.
- Memoize loader/normalizer safely. Bound fuzzy candidates and thresholds; log match source and score, not raw sample values.
- A dictionary match bypasses Gemini. A conflict or ambiguous entry goes to the ambiguity engine, not a forced first match.

3.4 Source collection/import script

- Create backend/scripts/import_headers.py only if no existing suitable importer exists.
- Read CSV columns: header, role optional, industry, language, source. Extend with source_url/verification fields if safe and compatible.
- Normalize and dedupe. Same phrase/same role merges provenance; same normalized phrase/different roles goes to conflict report. Only non-conflicting and approved entries merge into staged dictionary.
- Do not invent keywords or use AI to generate aliases. Do not scrape gated/private data. Do not treat “publicly viewable” as license approval; record source terms when applicable.

3.5 Evaluation script

- Create backend/scripts/eval_columns.py and a small golden CSV with header, expected_role, industry (optional), source and verification notes.
- Print overall accuracy, per-role precision/recall, top confusions, unresolved share, and proportion resolved by dictionary vs embedder vs Gemini-needed.

- Split test data into tuning vs held-out evaluation to prevent thresholds being tuned on the final test cases. Include every target industry where sufficient samples exist.
- Report false mappings alongside coverage. Increased coverage with worse precision is not a success.

Phase 3 exit tests

- Dictionary loader tests: Unicode, BOM, punctuation, case, Devanagari/Kannada preservation, exact match, bounded fuzzy, memoization and conflicting phrase behavior.
- Snapshot: every pre-migration alias resolves to same role.
- Import script: duplicate same-role merges; conflict different roles is reported; no auto-merge of conflicts.
- Eval script outputs documented metrics on golden test data.
- Dictionary-first routing, AI-off/embedder-unavailable routing, and “from dictionary / from similarity / from AI / from your earlier choice” UI source labels tested.
- All Phase 1 & Phase 2 tests and full backend/frontend suites pass. Provide report/ZIP and STOP for approval.

Phase 4 — Gemini leftovers, Check Again and custom notes

Start condition: explicit Phase 3 approval. Gemini improves convenience but is never required for safe core upload/analysis.

4.1 Check Again with AI

- Add button for still-unresolved columns. Force the Tier 2 request, bypassing the file-structure AI cache; apply a 10-second cooldown.
- Disable with a clear reason when AI is off or consent missing. Consent off must result in zero Gemini API calls.
- Show result, or the existing reason sentence, plus “AI looked at N columns, resolved M”. Do not fabricate N/M.
- Keep existing model names, fallback model, and timeouts from backend/.env or existing settings. Do not hardcode a new model or shrink timeouts arbitrarily.

4.2 Custom note per column

- User can enter a short note such as “this is the delivery charge”. Sanitize control characters and cap at 200 characters.
- One schema-bound Gemini request per file, maximum 20 columns, with header, masked samples and note. Treat note as untrusted data in a delimited field; instruct model to ignore instructions embedded inside the note.
- Allowed responses only: an existing allowed role, extra_dimension, extra_measure or ignore, plus a short reason. Strictly validate structure, enum and role; reject arbitrary text, code or invented formulas.
- Show result and require explicit user confirmation before applying. Money-sensitive roles (unit_price, cost, line_total, quantity) always need explicit confirmation.
- If AI off/fails, preserve note as display label and let user choose a role dropdown without blocking valid analysis.

4.3 Safety and privacy

- Never send values of customer/name-like or unidentified columns to Gemini. Reuse server-side pii_mask helpers; tests must inspect the actual request body.
- No eval, exec, dynamic Python, SQL construction from user headers, AI-generated code or formulas. Whitelist only.
- Missing/invalid key, unavailable model, invalid output or timeout must leave Phases 1–3 functional with a clear reason.
- Do not store custom note with sample values. Store only the approved mapping/note metadata using safe per-user ownership rules if persistence exists.

Phase 4 exit tests

- Mocked Gemini test for forced rerun and cache bypass.
- 10-second cooldown test; AI off/consent missing results in zero calls.
- Custom note flow: sanitization, truncation, delimited prompt, strict response validation, user confirmation gate.
- Invalid AI output rejected; money-sensitive roles require explicit confirmation.
- Request-body assertion proves customer values never appear in Gemini payload.
- Failure tests: missing/invalid key, timeout, fallback model failure and malformed response; Phases 1–3 stay working.
- All prior phase tests and complete backend/frontend suites pass. Provide report/ZIP and STOP for approval.

Dictionary evidence rules and attachment workflow

What counts as real-world evidence?

Evidence grade	Acceptable evidence	Allowed status
A — Observed official export/template	Exact header row from official downloadable template or an authorized export; file/module and source URL recorded.	Can be “observed-export”; role still needs semantic review.
B — Official docs field	Official documentation explicitly names and describes the field but no exact downloadable header was inspected.	“documented-field”; not falsely labelled observed-export.
C — Cited public dataset	Header extracted from cited/licensed raw public dataset; data dictionary reviewed.	“observed-dataset”; role may be candidate until reviewed.
D — Public community file	Public repository/Kaggle sample with traced provenance and terms.	“candidate-community”; must be reviewed before production.
E — User-confirmed header	User-provided export, with permission; collect headers only when possible.	“user-confirmed”; remove values and sensitive data.

How to handle the supplied starter pack

- Read verified_header_starter.csv, source_register.csv, README.md and IMPLEMENTATION_NOTES.md from the attached ZIP.
- Treat “proposed_role” as a suggestion, not a mandate. Some fields are context-dependent (for example Name, Amount, Rate, Order Number, Party Name, Location and Tax).
- Where source docs only describe fields but do not guarantee literal CSV header spelling, mark documented-field. Verify with exact downloadable templates if available.
- Compare source headers with existing aliases first. Do not duplicate same-role synonyms needlessly. Never overwrite conflicting existing roles without review.
- Keep starter pack and raw evidence inventory separate from production dictionary; only approved rows enter runtime lookup.

Official source links (research only as needed)

TallyPrime sample Excel templates: https://help.tallysolutions.com/download-sample-excel-files-with-data-for-import/

TallyPrime export data: https://help.tallysolutions.com/export-data-in-tally/

ERPNext Sales Invoice: https://docs.frappe.io/erpnext/sales-invoice

ERPNext Sales Order: https://docs.frappe.io/erpnext/sales-order

Shopify order exports: https://help.shopify.com/en/manual/fulfillment/managing-orders/exporting-orders

Shopify inventory CSV: https://help.shopify.com/en/manual/sell-in-person/shopify-pos/inventory-management/export-import-inventory-csv-pos

Zoho Books invoice documentation: https://www.zoho.com/us/books/help/invoice/

Zoho Books tax reports: https://www.zoho.com/us/books/help/reports/tax-reports.html

Odoo export/import documentation: https://www.odoo.com/documentation/17.0/applications/essentials/export_import_data.html

Square item library import/export: https://squareup.com/help/us/en/article/5153-import-items-online

UCI Machine Learning Repository: https://archive.ics.uci.edu/

Kaggle Datasets: https://www.kaggle.com/datasets

GitHub: https://github.com/

data.gov.in: https://www.data.gov.in/

Google Dataset Search: https://datasetsearch.research.google.com/

CBIC GST Invoice Rules: https://cbic-gst.gov.in/gst-invoice-rules.html

Use official/user-provided files first. These links are fallback research sources, not a checklist requiring every source to be downloaded. If an official template exists and is accessible, retrieve it; if access is unavailable, document it and continue.

What NOT to do — strict guardrails

- Do not invent keywords: no AI-generated aliases, fabricated translations, or synthetic “real” headers. Only observed/documented source-backed entries.
- Do not over-map: Country is not automatically Branch; StockCode is not automatically Item; Rate is not always Selling Price; Amount is not always line revenue; Name is not always customer/product.
- Do not change formulas: Pandas/NumPy own all calculations. Do not use Gemini to calculate Revenue, Cost, Profit, tax totals or create formulas.
- Do not treat unknown as zero: no missing Cost means no invented zero COGS/Profit. Show “Not available: needs X”.
- Do not reintroduce five required fields: upload confirmation requires only Date + a valid Amount path (Line Total OR Quantity and Selling Price).
- Do not send customer data: sample values from personal/name-like/unidentified columns must never reach Gemini; headers only in dictionary.
- Do not bypass consent or auth: Tier 2 calls require existing user consent. Mapping memory and custom notes must be owner-scoped.
- Do not blindly change fuzzy thresholds: validate against held-out files, precision/recall and false mappings first.
- Do not use the whole dictionary as Gemini context: local dictionary lookup remains offline-first and efficient.
- Do not bundle every phase in one implementation: only the currently approved phase can be changed.
- Do not claim tests pass without execution: report exact output. Never weaken/delete tests to make a suite green.
- Do not commit or push without approval: deliver changed-file ZIP if PR-writing access is unavailable or user wants local review.

Optional later work — not part of Phases 1–4

Weekly-off / closed-day detection is explicitly deferred until Phases 1–4 are green and approved. It must be separately scoped and tested; do not add it opportunistically during dictionary phases.

Acceptance criteria and final master prompt

Definition of done across all phases

- Each phase has a separate reviewable patch/PR or changed-files ZIP and CHANGES.md.
- All tests for previous approved phases remain green after the next phase.
- Backend full pytest suite and frontend full Vitest suite have actual recorded output; limitations are stated honestly.
- Existing alias snapshot remains unchanged unless a specific intentional behavior change is approved and documented.
- Existing electronics regression numbers are unchanged: Revenue 5,757,679; Cost 3,450,780; Profit 2,306,899; Units 2,621; Discount Given 1,211,000; Discount % 17.38.
- Date + Line Total and Date + Quantity + Selling Price each allow analysis without Category, Item or Cost Price.
- Missing optional concepts display unavailable/hidden states with reasons, never fabricated zero values.
- Every dictionary entry has provenance and review status. Conflicting or ambiguous entries are not automatically forced into calculations.
- Personal/unidentified values never appear in Gemini request bodies; AI consent logic, collision-first-wins, route/source, current model config and timeouts remain intact.
- Detection coverage, correct mapping, false mappings, unresolved rate and per-industry metrics are reported separately.

Copy-and-paste master prompt for Antigravity

You are a senior full-stack engineer working on SENOVA AI Dashboard. Read this entire PDF and all user-supplied attachments, including the real-world header starter pack ZIP, before making changes.

EXECUTION CONTROL: Work on exactly one phase at a time. Start with Phase 1 only. Do not implement or start Phase 2 until the user explicitly tests and approves Phase 1. After every phase, deliver the required patch/ZIP, CHANGES.md, exact commands and actual test output, then STOP and request approval. Do not use a fixed calendar timeline; move as soon as the current task and acceptance gate are complete, without rushing tests.

ATTACHMENTS-FIRST: Inventory and inspect supplied PDFs, ZIPs, CSV/XLSX/templates and source packs. Do not waste time re-downloading content already supplied. Use official source links only to fill documented gaps or verify exact header spellings. Do not claim an inaccessible file was inspected.

DICTIONARY: No AI-generated or guessed keywords. Collect only source-backed real-world headers. Keep source URL/path, industry/module, language, source type and verification status. Distinguish observed export header from a field merely named in documentation. Preserve raw spelling. Conflicts go to a review report; ambiguous entries are not force-mapped.

REQUIRED FIELDS: only Date plus one amount path: (A) Line Total; or (B) Quantity AND Selling Price. Category, Item, Cost Price, Discount, Tax, MRP, Stock On Hand and all other dimensions are optional for upload confirmation. Do not require the old fixed five fields. Missing optional roles degrade honestly (“Not available: needs X”); never replace unknown with zero.

CALCULATIONS/SAFETY: Pandas/NumPy perform all numbers. AI only suggests allowed column roles. Preserve order: offline dictionary/alias, bounded fuzzy as currently supported, FastEmbed, Gemini only for unresolved columns with explicit consent, privacy masking and allowed schema, then user note/skip. Preserve collision-first-wins. Never send values of customer/name-like or unidentified columns to Gemini. No eval/exec/dynamic code or formulas from AI/raw headers. Preserve current .env model configuration/timeouts.

REGRESSION: keep existing alias roles unchanged as proved by snapshot tests. Preserve electronics dataset results: Revenue 5757679, Cost 3450780, Profit 2306899, Units 2621, Discount Given 1211000, Discount % 17.38. Never weaken/delete tests.

PHASED QUALITY GATE: follow the current phase’s implementation and acceptance criteria in the PDF. Run relevant focused tests plus complete backend pytest and frontend Vitest suites before saying the phase is green. Record actual test counts and output, changed paths, source evidence, conflicts, coverage and remaining limitations. If tests cannot run, say so without claiming pass.

DELIVERY: If GitHub write/PR access works and is explicitly allowed, use a separate branch/PR per phase based on the previous approved phase. Otherwise create a ZIP containing only changed/new files at repository-relative paths, including CHANGES.md and apply instructions. Never commit/push/merge without permission.

BEGIN with audit and implementation for Phase 1 only. Do not ask the user to paste source files if repository/attachments are accessible. Do not stop just to ask questions for non-blocking ambiguities; choose the safest behavior, document it and continue. When Phase 1 passes, STOP for user approval.

End of playbook. This document sets implementation requirements and review gates; it does not claim any phase has already been implemented.